# Experiment 9

## Edit Distance (Levenshtein Distance) using Dynamic Programming

### Aim

To write a Java program to calculate the minimum Edit Distance (Levenshtein Distance) between two strings using Dynamic Programming, display the sequence of insertion, deletion, and replacement operations required to convert the first string into the second, and display the Dynamic Programming table constructed during the computation.

### Theory

Edit Distance, also known as Levenshtein Distance, is a string-processing problem that determines the minimum number of operations required to convert one string into another. The allowed operations are insertion, deletion, and replacement of a character, with each operation having a cost of one.

Dynamic Programming solves this problem by dividing it into smaller subproblems and storing their results in a two-dimensional table. Let the two strings be s1 and s2, with lengths $m$and $n$. The table dp[m+1][n+1] stores the minimum operations required to convert the first $i$characters of s1 into the first $j$characters of s2.

The first row and column are initialized to represent conversions involving an empty string. If the characters being compared are equal, the value is copied from the diagonal cell. Otherwise, the minimum cost among insertion, deletion, and replacement is calculated, and one is added. The final cell contains the minimum edit distance. By tracing backward through the table, the program reconstructs the sequence of operations.

- **Time complexity:** $O(mn)$
- **Space complexity:** $O(mn)$for the DP table.

### Algorithm

1. Read the two strings from the user.
2. Create a two-dimensional DP table of size (m + 1) × (n + 1).
3. Initialize the first row and first column with increasing values.
4. Compare each character of the two strings.
5. If the characters match, copy the value from the diagonal cell.
6. Otherwise, calculate the minimum cost of insertion, deletion, and replacement, and add one.
7. Display the completed DP table.
8. Start from the bottom-right cell and trace backward to identify the operations used.
9. Display the minimum edit distance and the sequence of operations required to convert the first string into the second.

### Program / Code

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter the first string: ");
String s1 = sc.nextLine();

System.out.print("Enter the second string: ");
String s2 = sc.nextLine();

int m = s1.length(), n = s2.length();
int[][] dp = new int[m + 1][n + 1];

for (int i = 0; i <= m; i++)
    dp[i][0] = i;

for (int j = 0; j <= n; j++)
    dp[0][j] = j;

for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (s1.charAt(i - 1) == s2.charAt(j - 1))
            dp[i][j] = dp[i - 1][j - 1];
        else
            dp[i][j] = 1 + Math.min(
                dp[i - 1][j - 1],
                Math.min(dp[i - 1][j], dp[i][j - 1])
            );
    }
}

System.out.println("\nDynamic Programming Table:");
System.out.print("    ");
for (int j = 0; j < n; j++)
    System.out.print(s2.charAt(j) + " ");
System.out.println();

for (int i = 0; i <= m; i++) {
    System.out.print((i == 0 ? '-' : s1.charAt(i - 1)) + " : ");
    for (int j = 0; j <= n; j++)
        System.out.print(dp[i][j] + " ");
    System.out.println();
}

List<String> operations = new ArrayList<>();
int i = m, j = n;

while (i > 0 || j > 0) {
    if (i > 0 && j > 0 &&
        s1.charAt(i - 1) == s2.charAt(j - 1)) {
        i--;
        j--;
    } else if (i > 0 && j > 0 &&
                dp[i][j] == dp[i - 1][j - 1] + 1) {
        operations.add("Replace '" + s1.charAt(i - 1)
            + "' with '" + s2.charAt(j - 1) + "'");
        i--;
        j--;
    } else if (i > 0 && dp[i][j] == dp[i - 1][j] + 1) {
        operations.add("Delete '" + s1.charAt(i - 1) + "'");
        i--;
    } else {
        operations.add("Insert '" + s2.charAt(j - 1) + "'");
        j--;
    }
}

Collections.reverse(operations);
```

```
        System.out.println("\nMinimum Edit Distance: " + dp[m][n]);
        System.out.println("Operations:");

        if (operations.isEmpty())
            System.out.println("No operations required.");
        else
            for (int k = 0; k < operations.size(); k++)
                System.out.println((k + 1) + ". " + operations.get(k));

        sc.close();
    }
}
```

## Input

```
Enter the first string: cat
Enter the second string: cut
```

## Output

```
Dynamic Programming Table:
    c u t
- : 0 1 2 3
c : 1 0 1 2
a : 2 1 1 2
t : 3 2 2 1

Minimum Edit Distance: 1
Operations:
1. Replace 'a' with 'u'
```

## Conclusion

The Edit Distance problem was successfully implemented in Java using Dynamic Programming. The program accepted two strings from the user, constructed a DP table, and calculated the minimum number of insertions, deletions, and replacements required to convert the first string into the second. It also displayed the completed table and reconstructed the sequence of operations by tracing backward through the table. For the sample strings cat and cut, the minimum edit distance was found to be 1, requiring only one replacement operation. Thus, the experiment demonstrated how Dynamic Programming avoids repeated calculations by storing intermediate results and efficiently solves string transformation problems with a time complexity of $O(mn)$.

## Learning Outcomes

- Understood the Edit Distance problem and the use of Dynamic Programming for string comparison.
- Learned to construct and initialize a two-dimensional DP table.
- Understood the recurrence relation for insertion, deletion, and replacement operations.
- Learned to reconstruct the minimum sequence of operations by backtracking through the DP table.
- Analysed the time and space complexity of the algorithm.
- Recognised the applications of Dynamic Programming in text comparison, spell checking, and string matching problems.